NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

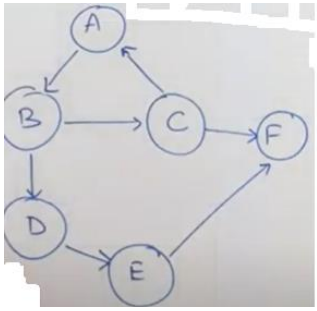
UCS 2312 Data Structures Lab

Assignment 9: Graph Traversal and its Applications
Date of Assignment: 14.11.2023

The cityADT consists of adjacency matrix that represents the connection between the cities. Adjacency matrix has an entry 1, if there is a connection between the cities. Implement the following methods. [CO2, K3]

- void create(cityADT *C) – will create the graph using adjacency matrix
- void disp(cityADT *C) – display the adjacency matrix
- void BFS(cityADT *C) – provides the output of visiting the cities by following breadth first
- void DFS(cityADT *C) – provides the output of visiting the cities by following depth first

1. Demonstrate the ADT with the following Graph



Enter the no. of vertices: 6
Enter the no. of edges: 7
AB, BC, BD, CA, CF, DE, EF

Adjacency Matrix

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	1	1	0	0
C	1	0	0	0	0	1
D	0	0	0	0	1	0
E	0	0	0	0	0	1
F	0	0	0	0	0	0

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

BFS Output: ABCDFE for Start vertex
DFS Output: ABCFDE for Start vertex A
CODE:
Graphadt.h

```
#include <stdio.h>
#include <stdlib.h>

struct graph
{
   int edges;int vertices;
   int arr[100][100];
};

struct edgepair
{
   int start;int end;
};

void init(struct graph *g,int n,int e)
{
   g->vertices=n;
   g->edges=e;
   for (int i=0;i<=g->vertices;i++)
   {   for (int j=0;j<=g->vertices;j++)
      {
         g->arr[i][j]=0;
      }
   }


}

void create(struct graph *g,struct edgepair *err[])
{
   for (int i=0;i<g->edges;i++)
   {
      g->arr[err[i]->start][err[i]->end]=1;
      //g->arr[err[i]->end][err[i]->start]=1; undirected graph

   }
   printf("done create\n");


}
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

```
void display(struct graph *g)
{
   for (int i=1;i<=g->vertices;i++)
   {   for (int j=1;j<=g->vertices;j++)
       {   printf("%d ",g->arr[i][j]);}
       printf("\n");

   }
}

struct queue
{
   int arr[100];
   int f,r;
   int size;
};

void initqueue(struct queue *h,int n)
{
   h->size=n;
   h->f=h->r=-1;
}
void enqueue(struct queue *h,int ele)
{
   h->r+=1;
   h->arr[h->r]=ele;
}

int isempty(struct queue *h)
{
   if (h->f==h->r)
       return 1;
   else
       return 0;
}

int dequeue(struct queue *h)
{
   h->f+=1;
  int d=h->arr[h->f];
  return d;
}

void bfs(struct graph *g,struct queue *q,int start)
{
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

```
  int visited[g->vertices+1];
  for (int i=0;i<=g->vertices;i++)
  {  visited[i]=0;  }
  enqueue(q,start);
  visited[1]=1;
  while (!isempty(q))
  {
     int z = dequeue(q);
     printf("%d",z);
     for (int i=1;i<=g->vertices;i++)
     {
        if (g->arr[z][i]==1 && visited[i]!=1)
        {
           visited[i]=1;
           enqueue(q,i);

        }

     }

  }
  printf("\n");
}

struct stack
{  int size;
  int top;
  int arr[100];
};

void createstack(struct stack *s,int size)
{
  s->size=size;
  s->top=-1;
}



void push(struct stack *s,int ele)
{
   s->top+=1;
  s->arr[s->top]=ele;
}

int isemptystack(struct stack *s)
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

```
{
   if (s->top==-1)
      return 1;
   else
      return 0;
}

void popstack(struct stack *s)
{
   s->top-=1;
}

int peek(struct stack *s)
{
    return s->arr[s->top];
}

void dfs(struct graph *g,struct stack *s1,int start)
{
   int visited[g->vertices+1];
   for (int i=0;i<=g->vertices;i++)
   {
      visited[i]=0;
   }
   push(s1,start);
   visited[start]=1;
   printf("%d",start);
   while (!isemptystack(s1))
   {
      int t=peek(s1);
      int flag =0;
      for (int i=1;i<=g->vertices;i++)
      {
         if (g->arr[t][i]==1 && visited[i]!=1)
         {
            flag =1;
            visited[i]=1;
            push(s1,i);
            printf("%d",i);
            break;
         }

      }
      if (flag == 0)
         popstack(s1);
```

```
    }
    printf("\n");
}
```

Graph.c

```
#include <stdio.h>
#include <stdlib.h>
#include "graphadt.h"

void main()
{
    struct graph *g;
    g=(struct graph *)malloc(sizeof(struct graph));
    printf("enter the number of vertices:");
    int n;
    scanf("%d",&n);
    printf("enter the number of edges:");
    int e;
    scanf("%d",&e);
    init(g,n,e);
    struct edgepair *err[e];
    for (int i=0;i<e;i++)
    {
        err[i]=(struct edgepair*)malloc(sizeof(struct edgepair));
        printf("EDGE PAIR %d\n",(i+1));
        printf("enter the start ");
        int s;
        scanf("%d",&s);
        err[i]->start=s;
        printf("enter the end ");
        int en;
        scanf("%d",&en);
        err[i]->end=en;

    }
    create(g,err);
    display(g);
    printf("\n---BFS---\n");
    struct queue *q;
    q=(struct queue*)malloc(sizeof(struct queue));
    initqueue(q,n);
    bfs(g,q,1);
    printf("\n---DFS---\n");
    struct stack *s;
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

```
  s=(struct stack *)malloc(sizeof(struct stack));
  createstack(s,n);
  dfs(g,s,1);
}
```

OUTPUT:

```
enter the number of vertices:6
enter the number of edges:7
EDGE PAIR 1
enter the start 1
enter the end 2
EDGE PAIR 2
enter the start 2
enter the end 3
EDGE PAIR 3
enter the start 2
enter the end 4
EDGE PAIR 4
enter the start 3
enter the end 1
EDGE PAIR 5
enter the start 3
enter the end 6
EDGE PAIR 6
enter the start 4
enter the end 5
EDGE PAIR 7
enter the start 5
enter the end 6
done create
0 1 0 0 0 0
0 0 1 1 0 0
1 0 0 0 0 1
0 0 0 0 1 0
0 0 0 0 0 1
0 0 0 0 0 0
```

```
---BFS---
123465

---DFS---
123645
```

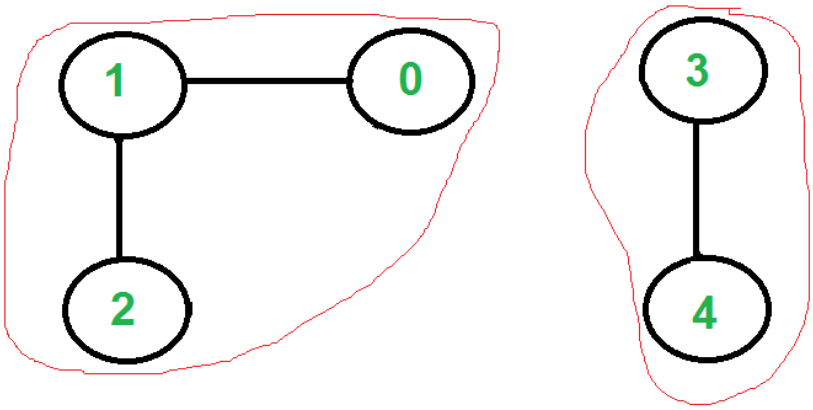
NAME: S.KARTHIKEYAN **3122 22 5001 056**
CSE B

2. Write an application to utilize traversals to do the following:

a. Given the source and destination cities, find whether there is a path from source to destination
b. Find the connected components in a given graph

Test the application with the following
Input:
Source: D
Destination: F
Output:
Path exists
Input:
Source: F
Destination: B
Output:
Path not exists

Input:



There are two connected components in above undirected graph
0 1 2
3 4

NAME: S.KARTHIKEYAN **3122 22 5001 056**

CSE B

ALGORITHM:

a)

Function Name: check

Input :

i)struct graph *

ii)struct stack *

iii)int start

iv)int end

Output:int

```
1. if (start==end)
       return 1;
2. int visited[g->vertices+1];
3. for (int i=0;i<=g->vertices;i++)
   {
     visited[i]=0;
   }
4.  push(s1,start);
5. visited[start]=1;
6. while (!isemptystack(s1))
   {
        int t=peek(s1);
       int flag =0;
       for (int i=1;i<=g->vertices;i++)
       {
       if (g->arr[t][i]==1 && visited[i]!=1)
       {
                       flag =1;
                       visited[i]=1;
                       push(s1,i);
                       if (end==i)
                               return 1;
                       else
                               break;
       }

       }
       if (flag == 0)
       popstack(s1);
   }
7. return 0;
```

NAME: S.KARTHIKEYAN **3122 22 5001 056**

CSE B

OUTPUT:

```
--CHECK IF PATH EXISTS--
enter start :1
enter end : 3
PATH EXISTS
```

ALGORITHM:

b)

Function name:Components

Input :

i)struct graph *

Output:Void

```
1. int visited[g->vertices+1];
2. for (int i=0;i<=g->vertices;i++)
3. visited[i]=0;
4. find(g,visited,1);
```

Function name: find

Input:

i)struct graph *

ii)int visited[]

iii)int x

```
1.  struct queue *q;
2. q=(struct queue*)malloc(sizeof(struct queue));
3.  initqueue(q,100);
4. enqueue(q,s);
5. visited[s]=1;
6. while (!isempty(q))
   {
    int z = dequeue(q);
    printf("%d",z);
   for (int i=1;i<=g->vertices;i++)
   {
   if (g->arr[z][i]==1 && visited[i]==0)
```

```
        {
                visited[i]=1;
                enqueue(q,i);
        }

        }

        }
```

```
7.  printf("\n");
8.  for (int i=1; i<=g->vertices; i++){
9.  if (visited[i]==0)
10. find(g,visited,i);
```

OUTPUT:

```
enter the start 1
enter the end 2
EDGE PAIR 2
enter the start 2
enter the end 3
EDGE PAIR 3
enter the start 4
enter the end 5
done create
0 1 0 0 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 1
0 0 0 0 0

---BFS---
123

---DFS---
123

--CHECK IF PATH EXISTS--
enter start :1
enter end : 2
PATH EXISTS

Components of the Graph
123
45
```

LEARNING OUTCOME:

Thus different graph traversal techniques have been learnt, understood and implemented.